Selenium 4 for Python

A practical guide with a worked example site

Prepared for Krish · selenium 4.51.0, headless Chrome 154, Python 3.13 · 14 October 2026

How to use this guide

1. Read Section 3. It shows the dummy site `DataJobs`. All examples use this site.
2. Read each example box. The upper part is the Python code. The lower part is the real output.
3. Use the *Targets* label. It gives the lines in Listing 1 (page 4) that the code uses.
4. Compare the *Before* and *After* screenshots of each interaction. Numbered boxes mark the elements.
5. Run the code yourself. Serve the folder `selenium-guide-files` (Section 3.1).
6. Use the cheat sheet (Section 15) as a quick reference.

Starting state of every example

Each example starts after this code. The examples do not repeat it.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import Select, WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

options = webdriver.ChromeOptions()
options.add_argument("--headless=new")
options.add_argument("--window-size=900,1000")
driver = webdriver.Chrome(options=options)
URL = "http://localhost:8000/index.html"
driver.get(URL)        # every example starts here, on a fresh page
```

All outputs come from a real run of the code against the dummy site. API facts follow the official documentation: https://www.selenium.dev/documentation/ and https://selenium-python.readthedocs.io/.

Contents

1 What Selenium is 1
2 Install and set up 1
3 The dummy website 3
4 Start and quit a driver 6
5 Open pages and move between them . . . 7
6 Locate elements 8
7 Read data from elements 10
8 Interact with the page 11
9 Waits 13
10 Dynamic content and JavaScript 16
11 Alerts, iframes and tabs 17
12 Screenshots 18
13 Scraping workflow: Selenium + pandas . . 19
14 Common pitfalls 20
15 Cheat sheet 22
A Glossary 23

1 What Selenium is

Selenium controls a real web browser from code. Your script opens pages, clicks, types and reads the result. The browser runs the JavaScript of the page. You see the page as a user sees it.

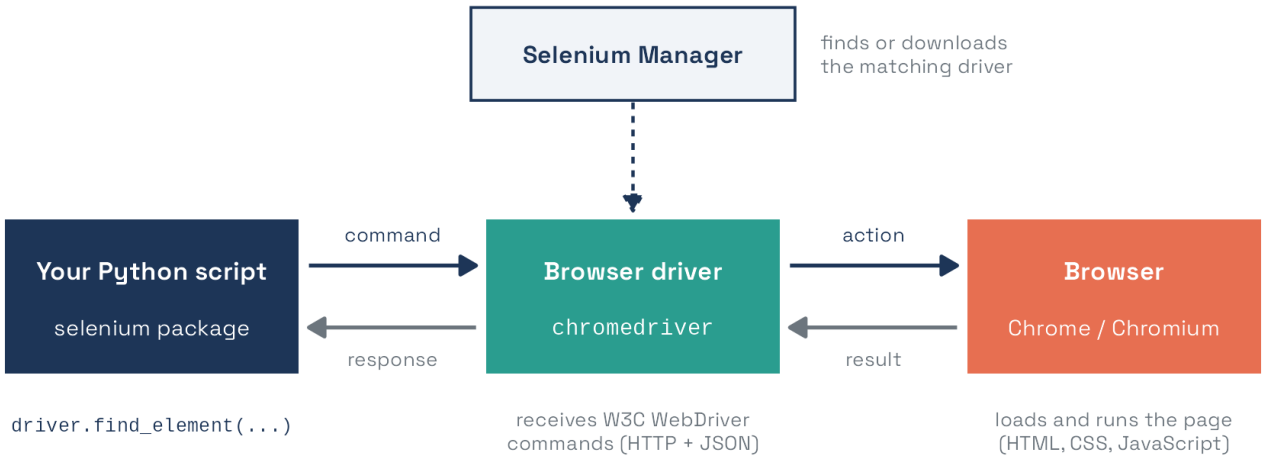


Figure 1. How Selenium works. Your script sends commands to a browser driver. The driver controls the browser. Selenium Manager supplies the driver.

1.1 When to use Selenium, and when not

	requests + BeautifulSoup	Selenium
What it gets	The HTML that the server sends	The page after the browser runs JavaScript
JavaScript	Not run	Run, as in a normal browser
Clicks, typing, forms	No (send HTTP requests yourself)	Yes
Speed	Fast (no browser)	Slow (starts a browser; renders each page)
Setup	`pip install requests beautifulsoup4`	`pip install selenium` and a browser
Use it for	Static pages, APIs, many pages	Pages that load data with JavaScript, logins, buttons, infinite scroll

Rule

Try `requests` first. Open the page source in your browser (Ctrl+U). If the data is in the source, you do not need Selenium. If the data appears only after JavaScript runs, use Selenium.

2 Install and set up

2.1 Install the package

1. Install a browser. This guide uses Google Chrome. Chromium, Firefox and Edge also work.
2. Install Selenium with pip. pip also installs its dependencies (`trio`, `urllib3`, `websocket-client` and others).
3. Do not download `chromedriver` yourself. Selenium Manager does it for you (see below).

```
pip install selenium
```

Output (last line, fresh virtual environment) — Targets: No page needed

```
Successfully installed attrs-26.1.0 certifi-2026.7.22 h11-0.16.0 idna-3.20 outcome-1.3.0.post0
↪  pysocks-1.7.1 selenium-4.51.0 sniffio-1.3.1 sortedcontainers-2.4.0 trio-0.34.0
↪  trio-websocket-0.12.2 typing_extensions-4.16.0 urllib3-2.8.0 websocket-client-1.9.2 wsproto-1.3.2
```

Verify the install. Selenium 4 does not have the old `find_element_by_id()` style methods:

```python
import selenium
print(selenium.__version__)
from selenium.webdriver.remote.webdriver import WebDriver
print(hasattr(WebDriver, "find_element_by_id"))   # old Selenium 3 API
```

Output — Targets: No page needed

```
4.51.0
False
```

2.2 Selenium Manager handles the driver

Selenium Manager ships with Selenium since version 4.6. When you start a driver and no driver is on your `PATH`, Selenium Manager:

1. finds the version of the installed browser;
2. downloads the matching driver (for example `chromedriver`);
3. stores the driver in the cache folder `~/.cache/selenium`.

If no browser is installed, Selenium Manager can also download one: Chrome for Testing (since 4.11), Firefox (4.12) and Edge (4.14). It needs internet access the first time.

Old tutorials

Old tutorials use `webdriver-manager` or `executable_path="chromedriver.exe"`. You do not need these in Selenium 4. They also use `driver.find_element_by_id("x")`. This method does not exist in Selenium 4. Use `driver.find_element(By.ID, "x")`.

3 The dummy website

`DataJobs` is a small job board. It has one main page (`index.html`) and three small pages: `about.html`, `salaries.html` and `newsletter.html` (shown inside an iframe). A JavaScript block at the end of `index.html` runs the search, the login form, the *Load more* button and the alert.

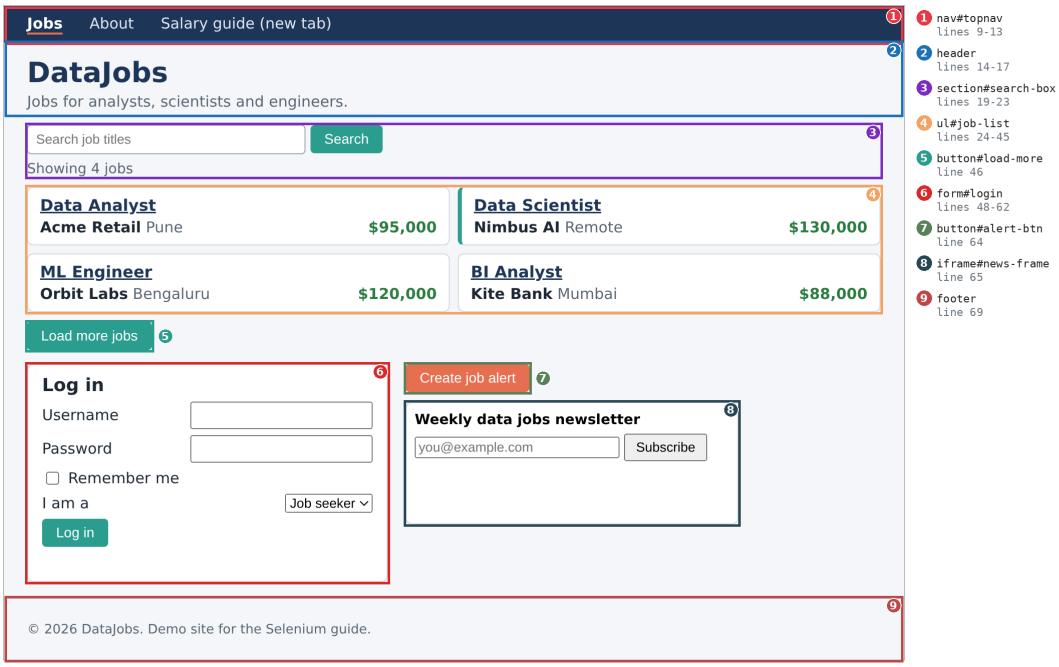


Figure 2. The page in headless Chrome. Numbered boxes mark the regions. The legend gives their lines in Listing 1.

Region	Lines	What it does
`nav#topnav`	9–13	Three links. *Salary guide* opens a new tab (`target="_blank"`).
`section#search-box`	19–23	Text box `#search`, button `#search-btn`, counter `#result-count`. JavaScript hides jobs that do not match.
`ul#job-list`	24–45	Four `li.job` cards (`id` job-1 to job-4, `data-salary`). job-2 also has class `remote`.
`button#load-more`	46	Adds three cards (job-5 to job-7, class `new`) after 1.5 seconds.
`form#login`	48–62	Username, password, checkbox, dropdown `#role`, submit button. Shows a message in `#login-msg`.
`button#alert-btn`	64	Opens a JavaScript alert.
`iframe#news-frame`	65	Shows `newsletter.html`: an email box and a *Subscribe* button.
`<script>`	70–114	The JavaScript for all behaviour above.

3.1 Serve the site

Selenium can open `file://` paths. A local web server is closer to a real site. Python has one built in:

```
cd selenium-guide-files
python -m http.server 8000
```

Output (the server keeps running; stop it with Ctrl+C) Targets: Serves every file in the folder

```
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Leave the terminal open. Open `http://localhost:8000/index.html` in a browser to check.

Listing 1. index.html (line numbers on the left)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>DataJobs - Jobs for data people</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<nav id="topnav">
  <a href="index.html" class="nav-link active">Jobs</a>
  <a href="about.html" class="nav-link">About</a>
  <a href="salaries.html" class="nav-link" target="_blank">Salary guide (new tab)</a>
</nav>
<header>
  <h1>DataJobs</h1>
  <p class="tagline">Jobs for analysts, scientists and engineers.</p>
</header>
<main>
<section id="search-box">
  <input type="text" id="search" name="q" placeholder="Search job titles">
  <button id="search-btn" type="button">Search</button>
  <p id="result-count">Showing 4 jobs</p>
</section>
<ul id="job-list">
  <li class="job" id="job-1" data-salary="95000">
    <h3 class="job-title"><a href="jobs/1.html">Data Analyst</a></h3>
    <span class="company">Acme Retail</span> <span class="location">Pune</span>
    <span class="salary">$95,000</span>
  </li>
  <li class="job remote" id="job-2" data-salary="130000">
    <h3 class="job-title"><a href="jobs/2.html">Data Scientist</a></h3>
    <span class="company">Nimbus AI</span> <span class="location">Remote</span>
    <span class="salary">$130,000</span>
  </li>
  <li class="job" id="job-3" data-salary="120000">
    <h3 class="job-title"><a href="jobs/3.html">ML Engineer</a></h3>
    <span class="company">Orbit Labs</span> <span class="location">Bengaluru</span>
    <span class="salary">$120,000</span>
  </li>
  <li class="job" id="job-4" data-salary="88000">
    <h3 class="job-title"><a href="jobs/4.html">BI Analyst</a></h3>
    <span class="company">Kite Bank</span> <span class="location">Mumbai</span>
    <span class="salary">$88,000</span>
  </li>
</ul>
<button id="load-more" type="button">Load more jobs</button>
<section id="account">
  <form id="login" action="#">
    <h2>Log in</h2>
    <label>Username <input type="text" id="username" name="username"></label>
    <label>Password <input type="password" id="password" name="password"></label>
    <label class="check"><input type="checkbox" id="remember" name="remember"> Remember me</label>
    <label>I am a
      <select id="role" name="role">
        <option value="seeker">Job seeker</option>
        <option value="recruiter">Recruiter</option>
        <option value="student">Student</option>
      </select>
    </label>
    <button type="submit" id="login-btn">Log in</button>
    <p id="login-msg"></p>
  </form>
  <div id="extras">
    <button id="alert-btn" type="button">Create job alert</button>
    <iframe id="news-frame" name="news" src="newsletter.html" title="Newsletter"></iframe>
  </div>
</section>
</main>
<footer><p>&copy; 2026 DataJobs. Demo site for the Selenium guide.</p></footer>
<script>
const MORE = [["Data Engineer", "Delta Logistics", "Chennai", 110000],
              ["Research Scientist", "Quanta Health", "Remote", 140000],
              ["Analytics Manager", "Kite Bank", "Mumbai", 125000]];
function count() {
  const shown = [...document.querySelectorAll("li.job")].filter(li => li.style.display !== "none");
  document.getElementById("result-count").textContent = "Showing " + shown.length + " jobs";
}
```

```
document.getElementById("search-btn").addEventListener("click", () => {
  const q = document.getElementById("search").value.toLowerCase();
  document.querySelectorAll("li.job").forEach(li => {
    const t = li.querySelector(".job-title").textContent.toLowerCase();
    li.style.display = t.includes(q) ? "" : "none";
  });
  count();
});
document.getElementById("search").addEventListener("keydown", e => {
  if (e.key === "Enter") document.getElementById("search-btn").click();
});
document.getElementById("load-more").addEventListener("click", e => {
  e.target.disabled = true; e.target.textContent = "Loading...";
  setTimeout(() => {                        // simulate a slow server: 1.5 s
    const ul = document.getElementById("job-list");
    MORE.forEach(([title, co, loc, pay], i) => {
      const li = document.createElement("li");
      li.className = "job new"; li.id = "job-" + (5 + i); li.dataset.salary = pay;
      li.innerHTML = `<h3 class="job-title"><a href="jobs/${5 + i}.html">${title}</a></h3>
        <span class="company">${co}</span> <span class="location">${loc}</span>
        <span class="salary">$${pay.toLocaleString("en-US")}</span>`;
      ul.appendChild(li);
    });
    e.target.textContent = "No more jobs"; count();
  }, 1500);
});
document.getElementById("login").addEventListener("submit", e => {
  e.preventDefault();
  const user = document.getElementById("username").value;
  const role = document.getElementById("role");
  const msg = user ? "Welcome, " + user + " (" + role.selectedOptions[0].text + ")" : "Enter a username";
  document.getElementById("login-msg").textContent = msg;
});
document.getElementById("alert-btn").addEventListener("click", () => {
  alert("Job alert saved for: data jobs");
});
</script>
</body>
</html>
```

Listing 2. newsletter.html (loaded in the iframe)

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Newsletter</title>
<style>body{font-family:"DejaVu Sans",Arial,sans-serif;margin:8px;font-size:14px} button{padding:4px
↪  10px}</style></head>
<body>
  <h4 id="news-title" style="margin:0 0 6px">Weekly data jobs newsletter</h4>
  <input type="email" id="email" placeholder="you@example.com">
  <button id="subscribe" type="button"
          onclick="document.getElementById('news-msg').textContent='Subscribed:
          ↪  '+document.getElementById('email').value">Subscribe</button>
  <p id="news-msg"></p>
</body>
</html>
```

4 Start and quit a driver

4.1 Start, check, quit

1. Make a `ChromeOptions` object.
2. Add the options you need.
3. Make the driver with `webdriver.Chrome(options=...)`. This starts the browser.
4. Call `driver.quit()` at the end. This closes all windows and stops the driver process.

```
from selenium import webdriver

options = webdriver.ChromeOptions()
options.add_argument("--headless=new")      # no visible window
options.add_argument("--window-size=1200,800")
d = webdriver.Chrome(options=options)        # Selenium Manager finds chromedriver
print(d.capabilities["browserName"], d.capabilities["browserVersion"])
print(d.get_window_size())
d.quit()                                     # closes the browser and the driver
```

Output — Targets: No page needed

```
chrome 154.0.8037.57
{'width': 1200, 'height': 800}
```

Use `try`/`finally`. Then the browser closes also when your code raises an error:

```
d = webdriver.Chrome(options=options)
try:
    d.get(URL)
    print(d.title)
finally:
    d.quit()            # runs even if the code above fails
```

Output — Targets: index.html line 5

```
DataJobs - Jobs for data people
```

4.2 Common options

Code	Effect
`options.add_argument("--headless=new")`	Runs Chrome with no visible window. Use it on servers and in notebooks.
`options.add_argument("--window-size=1200,800")`	Sets the window size. Set it in headless mode (see Section 14).
`options.page_load_strategy = "eager"`	`get()` returns when the HTML is parsed, not when all images load. Default: `"normal"`.
`options.binary_location = "/path/to/chrome"`	Uses a specific browser binary, for example Chromium.
`webdriver.Firefox()`, `webdriver.Edge()`	Other browsers. Each has its own options class.

Headless Chrome or Chromium

Use `--headless=new`. Chrome's new headless mode is the same browser as normal Chrome, with no window. On Linux, Chromium works the same way. Set `binary_location` if Selenium does not find it.

5 Open pages and move between them

Code	Effect
`driver.get(url)`	Opens the URL. Waits until the page load completes (`document.readyState` is `"complete"`).
`driver.title`, `driver.current_url`	The title and URL of the current page.
`driver.page_source`	The current HTML of the page, after JavaScript changes.
`driver.back()`, `forward()`, `refresh()`	The browser buttons.

```
print(driver.title)
print(driver.current_url)
print(len(driver.page_source), "characters of HTML")
```

Output — Targets: index.html line 5

```
DataJobs - Jobs for data people
http://localhost:8000/index.html
4946 characters of HTML
```

```
driver.find_element(By.LINK_TEXT, "About").click()
print("1", driver.title)
driver.back()
print("2", driver.title)
driver.forward()
print("3", driver.title, driver.current_url)
driver.refresh()
print("4", driver.title)
```

Output — Targets: index.html lines 5, 11

```
1 About DataJobs
2 DataJobs - Jobs for data people
3 About DataJobs http://localhost:8000/about.html
4 About DataJobs
```

`get()` does not wait for JavaScript data

`get()` waits for the page load. It does not wait for data that JavaScript loads later. Use an explicit wait for that data (Section 9).

6 Locate elements

6.1 The eight locator strategies

Import `By` from the module `selenium.webdriver.common.by`. Each call takes a strategy and a value. Example: `driver.find_element(By.ID, "search")`.

Strategy	Example value	Matches
`By.ID`	`"search"`	the element with `id="search"`
`By.NAME`	`"username"`	elements with `name="username"` (form fields)
`By.CLASS_NAME`	`"job"`	elements that have the class `job` (one class only)
`By.TAG_NAME`	`"h1"`	elements with that tag
`By.CSS_SELECTOR`	`"#job-list li.job .salary"`	elements that match the CSS selector
`By.XPATH`	`"//li[@data-salary > 100000]//a"`	nodes that match the XPath expression
`By.LINK_TEXT`	`"About"`	`<a>` elements whose visible text is exactly the value
`By.PARTIAL_LINK_TEXT`	`"Salary"`	`<a>` elements whose visible text contains the value

```
box = driver.find_element(By.ID, "search")
print(box.tag_name, box.get_attribute("placeholder"))
user = driver.find_element(By.NAME, "username")
print(user.get_attribute("id"))
```

Output — Targets: index.html lines 20, 50

```
input Search job titles
username
```

```
jobs = driver.find_elements(By.CLASS_NAME, "job")
print(len(jobs), "elements with class job")
h1 = driver.find_element(By.TAG_NAME, "h1")
print(h1.text)
```

Output — Targets: index.html lines 15, 25--44

```
4 elements with class job
DataJobs
```

```
salaries = driver.find_elements(By.CSS_SELECTOR, "#job-list li.job .salary")
print([s.text for s in salaries])
remote = driver.find_element(By.CSS_SELECTOR, "li.job.remote h3 a")
print(remote.text)
```

Output — Targets: index.html lines 28, 31, 33, 38, 43

```
['$95,000', '$130,000', '$120,000', '$88,000']
Data Scientist
```

```
a = driver.find_element(By.XPATH, "//li[@data-salary > 100000]//a")
print(a.text)
names = driver.find_elements(By.XPATH, "//span[@class='company']")
print([n.text for n in names])
```

Output — Targets: index.html lines 27, 31--32, 37, 42

```
Data Scientist
['Acme Retail', 'Nimbus AI', 'Orbit Labs', 'Kite Bank']
```

```
link = driver.find_element(By.LINK_TEXT, "About")
print(link.get_attribute("href"))
link2 = driver.find_element(By.PARTIAL_LINK_TEXT, "Salary")
print(link2.text)
```

Output Targets: index.html lines 10--12

```
http://localhost:8000/about.html
Salary guide (new tab)
```

6.2 `find_element` or `find_elements`

	`find_element`	`find_elements`
Returns	the first match (one `WebElement`)	all matches (a list)
No match	raises `NoSuchElementException`	returns `[]`
Use it to	get one element you expect	count, loop, or test if something exists

```
print(driver.find_elements(By.CLASS_NAME, "no-such-class"))   # empty list
driver.find_element(By.CLASS_NAME, "no-such-class")           # raises
```

Output (raises an error; message cut to its first line) Targets: nothing (the class does not exist)

```
[]
NoSuchElementException: no such element: Unable to locate element: {"method":"css
↪  selector","selector":".no-such-class"}
```

Call `find_element` on an element to search only inside it:

```
card = driver.find_element(By.ID, "job-3")
print(card.find_element(By.CLASS_NAME, "company").text)
print(card.find_element(By.CLASS_NAME, "salary").text)
```

Output Targets: index.html lines 35--39

```
Orbit Labs
$120,000
```

6.3 Choose a locator

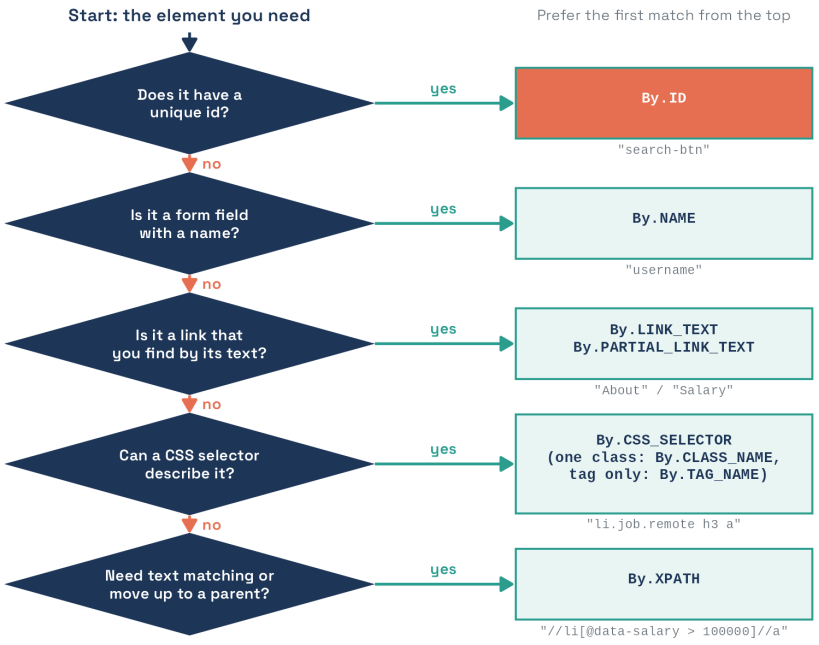


Figure 3. Locator decision chart. Start at the top. Use the first strategy that fits. Values below the boxes come from the dummy site.

7 Read data from elements

Code	Returns
`el.text`	The visible text, as the user sees it. Hidden elements give `""`.
`el.get_attribute(name)`	The property with that name if it exists, else the HTML attribute, else `None`.
`el.get_dom_attribute(name)`	The HTML attribute exactly as written, else `None`.
`el.get_property(name)`	The JavaScript property (for example `value` of a text box).
`el.tag_name`	The tag name, for example `"input"`.
`el.is_displayed()`	`True` if the user can see the element.
`el.is_enabled()`	`False` for a disabled control.
`el.is_selected()`	`True` for a ticked checkbox, a chosen radio button or option.

```
card = driver.find_element(By.ID, "job-2")
print(repr(card.text))
```

Output — Targets: index.html lines 30--34

```
'Data Scientist\nNimbus AI Remote\n$130,000'
```

```
a = driver.find_element(By.CSS_SELECTOR, "#job-1 a")
print(a.get_attribute("href"))        # property value: full URL
print(a.get_dom_attribute("href"))    # exactly as written in the HTML
card = driver.find_element(By.ID, "job-1")
print(card.get_attribute("data-salary"), card.get_attribute("class"))
print(repr(card.get_attribute("title")))      # no such attribute, but a title property exists
print(repr(card.get_dom_attribute("title")))  # no such attribute in the HTML
```

Output — Targets: index.html lines 25--29

```
http://localhost:8000/jobs/1.html
jobs/1.html
95000 job
''
None
```

```
remember = driver.find_element(By.ID, "remember")
print("displayed", remember.is_displayed())
print("enabled  ", remember.is_enabled())
print("selected ", remember.is_selected())
student = driver.find_element(By.CSS_SELECTOR, "#role option[value=student]")
print("student option selected", student.is_selected())
```

Output — Targets: index.html lines 52, 54--58

```
displayed True
enabled   True
selected  False
student option selected False
```

8 Interact with the page

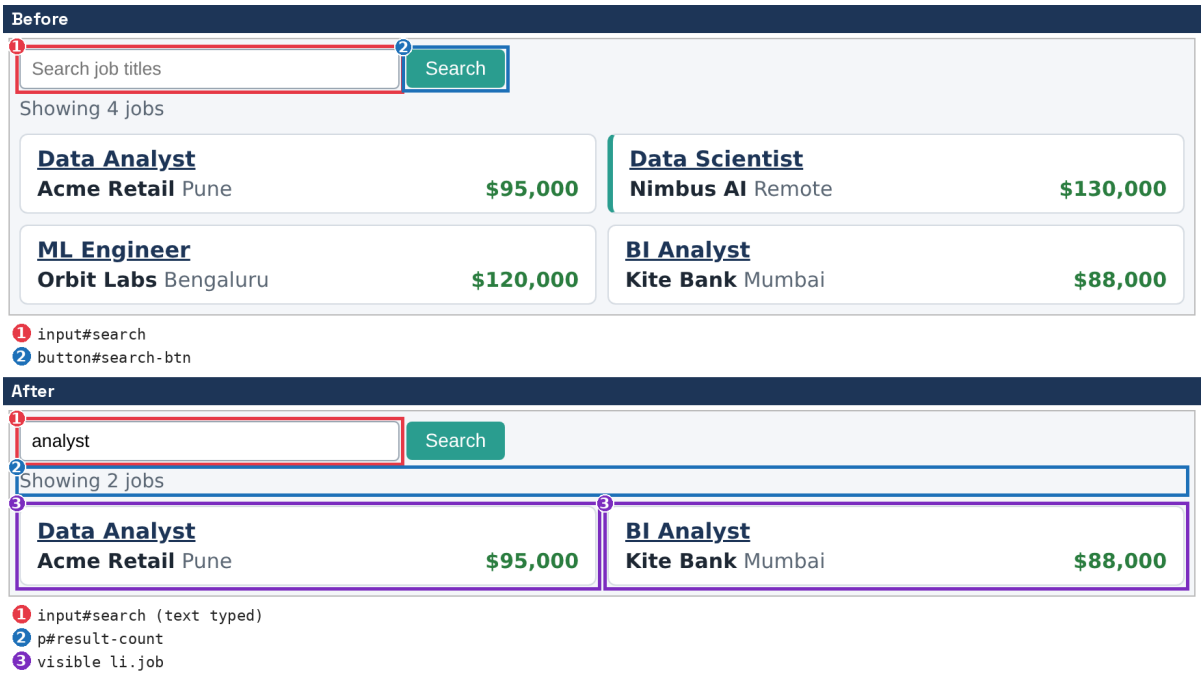
8.1 Type and click

1. Find the text box.
2. Type with `send_keys()`.
3. Find the button.
4. Click it with `click()`.
5. Read the result.

```
box = driver.find_element(By.ID, "search")
box.send_keys("analyst")
driver.find_element(By.ID, "search-btn").click()
print(driver.find_element(By.ID, "result-count").text)
titles = driver.find_elements(By.CSS_SELECTOR, ".job-title a")
print([t.text for t in titles if t.is_displayed()])
```

Output — Targets: index.html lines 19--23, 25--44

```
Showing 2 jobs
['Data Analyst', 'BI Analyst']
```



Use `clear()` to empty a field. Use `Keys` for special keys. Import it from `selenium.webdriver.common.keys`.

```
box = driver.find_element(By.ID, "search")
box.send_keys("data")
print(repr(box.get_attribute("value")))
box.clear()
print(repr(box.get_attribute("value")))
box.send_keys("scientist", Keys.ENTER)       # Enter key runs the search
print(driver.find_element(By.ID, "result-count").text)
```

Output — Targets: index.html lines 20, 22

```
'data'
''
Showing 1 jobs
```

Key	Use
`Keys.ENTER`, `Keys.RETURN`	Submit a search or a form field
`Keys.TAB`, `Keys.ESCAPE`, `Keys.BACKSPACE`	Move to the next field; close a popup; delete one character
`Keys.ARROW_DOWN`, `Keys.PAGE_DOWN`, `Keys.END`, `Keys.HOME`	Move in lists and pages
`Keys.CONTROL + "a"`	Key combination (select all)

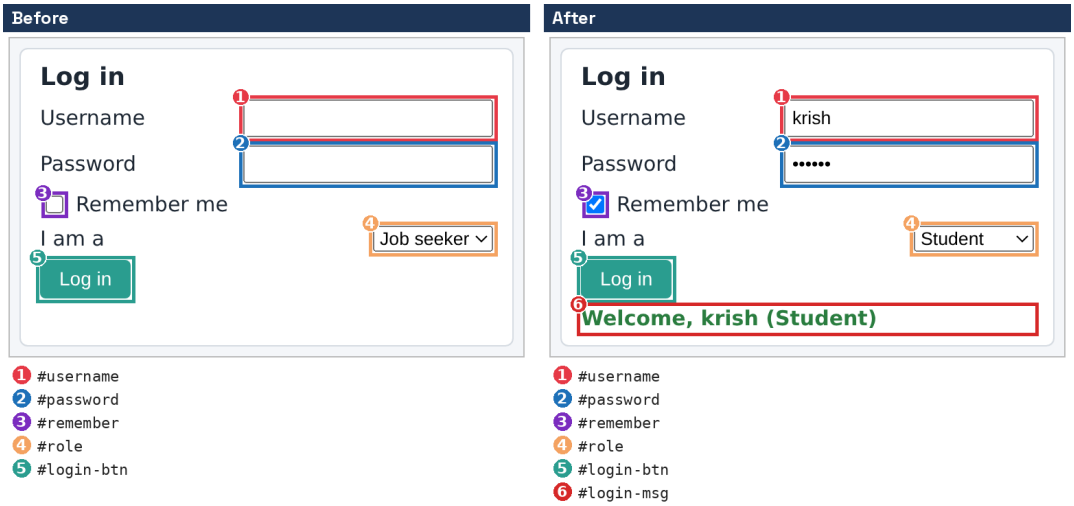
8.2 Fill a form: text, checkbox, dropdown, submit

A checkbox is an `<input>`. Click it to tick or untick it. A dropdown is a `<select>`. Wrap it in `Select` (from `selenium.webdriver.support.ui`).

```
driver.find_element(By.ID, "username").send_keys("krish")
driver.find_element(By.ID, "password").send_keys("s3cret")
driver.find_element(By.ID, "remember").click()         # tick the checkbox
Select(driver.find_element(By.ID, "role")).select_by_visible_text("Student")
driver.find_element(By.ID, "login-btn").click()
print(driver.find_element(By.ID, "login-msg").text)
print(driver.find_element(By.ID, "remember").is_selected())
```

Output — Targets: index.html lines 48--62

```
Welcome, krish (Student)
True
```



Dropdowns with `Select`

```
role = Select(driver.find_element(By.ID, "role"))
print([o.text for o in role.options])
print(role.first_selected_option.text)       # default choice
role.select_by_value("recruiter")
print(role.first_selected_option.text)
role.select_by_index(2)
print(role.first_selected_option.get_attribute("value"))
```

Output — Targets: index.html lines 54--58

```
['Job seeker', 'Recruiter', 'Student']
Job seeker
Recruiter
student
```

Submit without the button

`submit()` submits the form that contains the element:

```
user = driver.find_element(By.ID, "username")
user.send_keys("krish")
user.submit()                    # submits the form that contains the field
print(driver.find_element(By.ID, "login-msg").text)
```

Output Targets: index.html lines 50, 61

```
Welcome, krish (Job seeker)
```

9 Waits

JavaScript can add elements after the page loads. Your code can run before they exist. The *Load more jobs* button (line 46) adds three jobs after 1.5 seconds.

9.1 The problem

```
driver.find_element(By.ID, "load-more").click()
driver.find_element(By.ID, "job-5")          # not there yet
```

Output (raises an error; message cut to its first line) Targets: index.html line 46 (button); job-5 is added by the script, lines 89--103

```
NoSuchElementException: no such element: Unable to locate element: {"method":"css
↪  selector","selector":"[id="job-5"]"}
```

9.2 Three ways to wait

Method	How it works	Problem
`time.sleep(3)`	Always stops for 3 s	Too short: fails. Too long: slow. You guess.
`driver.implicitly_wait(5)`	Every `find_element` retries for up to 5 s. Default 0.	Global. Waits only for *presence*, not visibility or clickability.
`WebDriverWait(driver, 5)`	Checks one condition, every 0.5 s, for up to 5 s	None. Use it.

```
start = time.perf_counter()
driver.find_element(By.ID, "load-more").click()
time.sleep(3)                                # a guess
print(driver.find_element(By.ID, "job-5").text.splitlines()[0])
print(f"waited {time.perf_counter() - start:.1f} s")
```

Output Targets: button line 46; new li.job added by script lines 89--103

```
Data Engineer
waited 3.0 s
```

```
driver.implicitly_wait(5)          # applies to every find_element call
start = time.perf_counter()
driver.find_element(By.ID, "load-more").click()
print(driver.find_element(By.ID, "job-5").text.splitlines()[0])
print(f"waited {time.perf_counter() - start:.1f} s")
driver.implicitly_wait(0)          # back to the default
```

Output Targets: button line 46; new li.job added by script lines 89--103

```
Data Engineer
waited 1.6 s
```

Do not mix implicit and explicit waits

The Selenium documentation warns: mixing them can cause unpredictable wait times. Keep the implicit wait at 0. Use explicit waits.

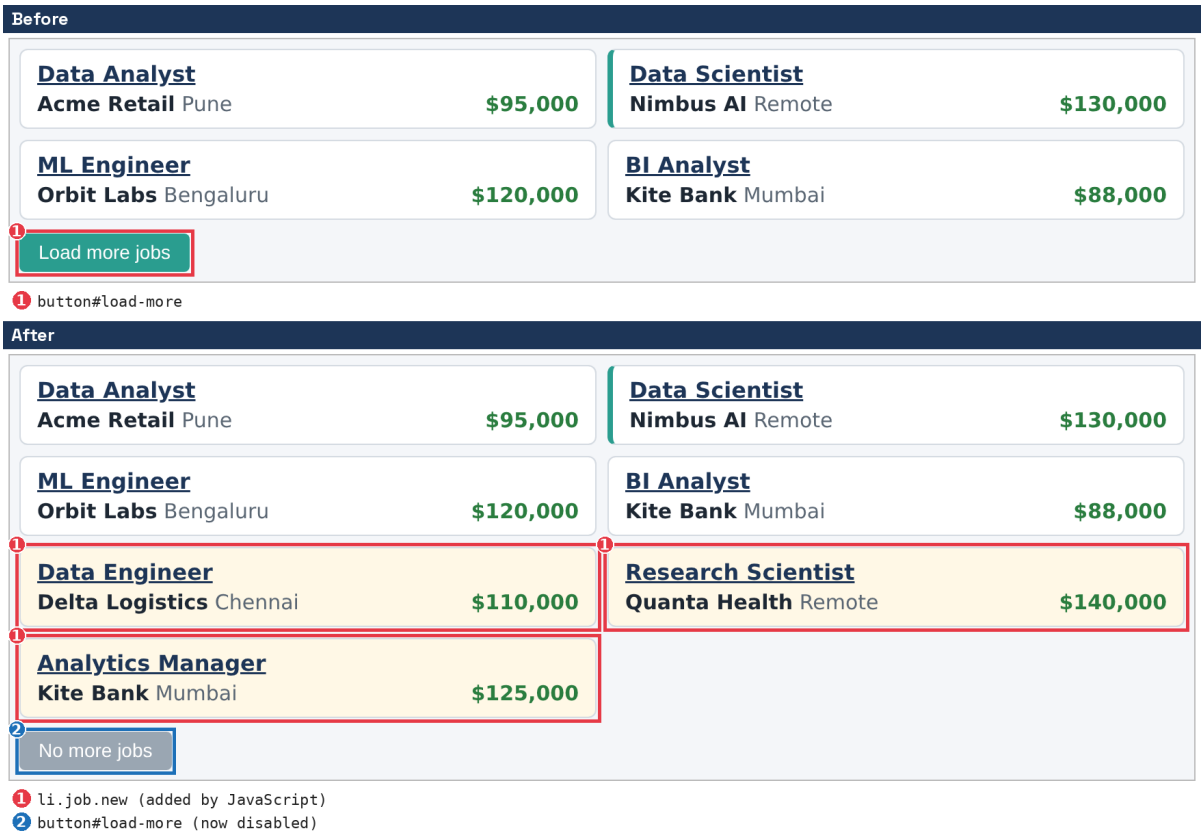
9.3 Explicit waits with expected conditions

1. Make a `WebDriverWait` with a timeout in seconds.
2. Call `until()` with a condition.
3. Use the return value. For element conditions, it is the element.

```
driver.find_element(By.ID, "load-more").click()
start = time.perf_counter()
wait = WebDriverWait(driver, timeout=5)
job7 = wait.until(EC.visibility_of_element_located((By.ID, "job-7")))
print(job7.text.splitlines()[0])
print(f"waited {time.perf_counter() - start:.1f} s")
print(len(driver.find_elements(By.CSS_SELECTOR, "li.job")), "jobs")
```

Output — Targets: button line 46; new li.job added by script lines 89--103

```
Analytics Manager
waited 1.5 s
7 jobs
```




```
driver.find_element(By.ID, "load-more").click()
wait = WebDriverWait(driver, 5)
ok = wait.until(EC.text_to_be_present_in_element((By.ID, "result-count"), "7"))
print(ok, "|", driver.find_element(By.ID, "result-count").text)
btn = driver.find_element(By.ID, "load-more")
print(btn.text, "| enabled:", btn.is_enabled())
```

Output — Targets: index.html lines 22, 46

```
True | Showing 7 jobs
No more jobs | enabled: False
```

Your own condition

`until()` accepts any function of the driver. It returns the first truthy value. An empty list is not truthy, so the wait continues.

```
driver.find_element(By.ID, "load-more").click()
wait = WebDriverWait(driver, 5, poll_frequency=0.2)
jobs = wait.until(lambda d: d.find_elements(By.CSS_SELECTOR, "li.job.new"))
print(len(jobs), "new jobs")
```

Output — Targets: li.job.new, added by script lines 89--103

```
3 new jobs
```

When the time runs out

```
driver.find_element(By.ID, "load-more").click()
try:
    WebDriverWait(driver, 1).until(
        EC.presence_of_element_located((By.ID, "job-5")))
except TimeoutException:
    print("TimeoutException after 1 s: jobs need 1.5 s")
```

Output — Targets: li#job-5, added after 1.5 s (script lines 89--103)

```
TimeoutException after 1 s: jobs need 1.5 s
```

Condition (`EC. ...`)	True when
`presence_of_element_located(loc)`	the element is in the DOM (it may be hidden)
`visibility_of_element_located(loc)`	the element is in the DOM and visible
`element_to_be_clickable(loc)`	the element is visible and enabled
`presence_of_all_elements_located(loc)`	at least one element matches
`text_to_be_present_in_element(loc, text)`	the element's text contains `text`
`invisibility_of_element_located(loc)`	the element is hidden or gone (spinners)
`staleness_of(el)`	the element is removed from the DOM
`title_contains(text)`, `url_contains(text)`	the title or URL contains `text`
`alert_is_present()`	an alert is open; returns the alert
`frame_to_be_available_and_switch_to_it(loc)`	the frame exists; Selenium switches into it
`number_of_windows_to_be(n)`	`n` windows or tabs are open

`loc` is a tuple: `(By.ID, "job-7")`. Pass the tuple as one argument; note the double brackets.

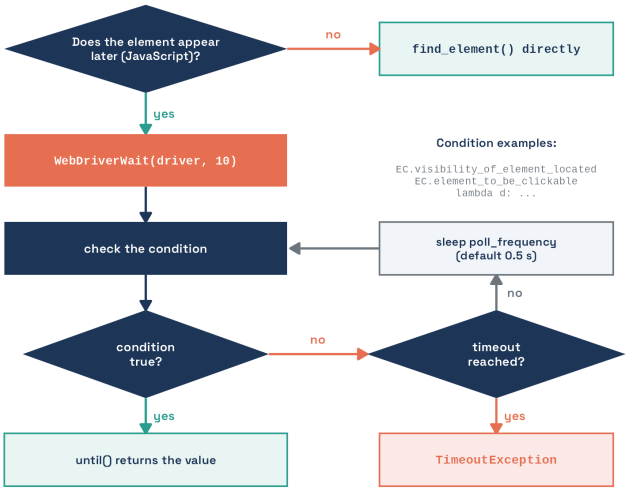


Figure 4. Wait strategy. `WebDriverWait` checks the condition until it is true or the timeout passes.

10 Dynamic content and JavaScript

10.1 Run JavaScript

`driver.execute_script(code, *args)` runs JavaScript in the page. Use `return` to get a value back. Elements you pass appear as `arguments[0]`, `arguments[1]`, …

```
n = driver.execute_script("return document.querySelectorAll('li.job').length")
print(n, type(n).__name__)
card = driver.find_element(By.ID, "job-4")
print(driver.execute_script("return arguments[0].dataset.salary", card))
print(driver.execute_script("return [window.innerWidth, window.innerHeight]"))
```

Output — Targets: index.html lines 25--44

```
4 int
88000
[900, 913]
```

10.2 Scroll

Some pages load more items when you scroll. Scroll with JavaScript, then wait for the new items.

```
driver.set_window_size(900, 400)              # small window: page needs scrolling
print(driver.execute_script("return window.scrollY"))
driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")
print(driver.execute_script("return window.scrollY"))
btn = driver.find_element(By.ID, "alert-btn")
driver.execute_script("window.scrollTo(0, 0)")
driver.execute_script("arguments[0].scrollIntoView({block: 'center'})", btn)
print(driver.execute_script("return window.scrollY"))
```

Output — Targets: index.html line 64

```
0
341
216
```

The Actions API can also scroll to an element:

```
from selenium.webdriver import ActionChains
driver.set_window_size(900, 400)
footer = driver.find_element(By.TAG_NAME, "footer")
ActionChains(driver).scroll_to_element(footer).perform()
print(driver.execute_script(
    "const r = arguments[0].getBoundingClientRect(); return r.bottom <= window.innerHeight", footer))
```

Output — Targets: index.html line 69

```
True
```

Pattern for infinite scroll

1. Count the items.
2. Scroll to the bottom.
3. Wait until the count grows (`WebDriverWait` with a lambda).
4. Stop when the count stays the same (catch `TimeoutException`).

11 Alerts, iframes and tabs

11.1 Alerts

A JavaScript alert blocks the page. Switch to it, read it, then accept or dismiss it.

```
driver.find_element(By.ID, "alert-btn").click()
alert = WebDriverWait(driver, 3).until(EC.alert_is_present())
print(alert.text)
alert.accept()                    # press OK (dismiss() presses Cancel)
print(driver.title)               # page is usable again
```

Output — Targets: index.html line 64

```
Job alert saved for: data jobs
DataJobs - Jobs for data people
```

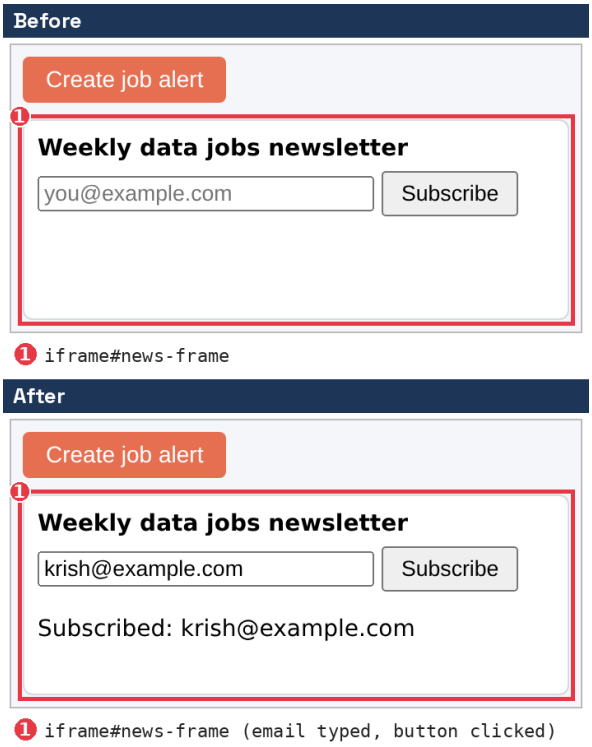
11.2 Iframes

An iframe holds a separate page. Selenium searches only the current page. Switch into the frame first. Switch back after.

```
try:
    driver.find_element(By.ID, "email")          # inside the iframe
except NoSuchElementException:
    print("not found from the main page")
driver.switch_to.frame("news")                    # name or id of the iframe
driver.find_element(By.ID, "email").send_keys("krish@example.com")
driver.find_element(By.ID, "subscribe").click()
print(driver.find_element(By.ID, "news-msg").text)
driver.switch_to.default_content()                # back to the main page
print(driver.find_element(By.TAG_NAME, "h1").text)
```

Output — Targets: index.html line 65 (iframe); newsletter.html lines 7--10

```
not found from the main page
Subscribed: krish@example.com
DataJobs
```



11.3 Windows and tabs

Each window or tab has a handle (a string). Selenium stays on the old tab until you switch.

```
main = driver.current_window_handle
driver.find_element(By.PARTIAL_LINK_TEXT, "Salary guide").click()
WebDriverWait(driver, 3).until(EC.number_of_windows_to_be(2))
new = [h for h in driver.window_handles if h != main][0]
driver.switch_to.window(new)
print(driver.title)
print(driver.find_element(By.ID, "salary-table").text.splitlines()[1])
driver.close()                    # closes this tab only
driver.switch_to.window(main)
print(driver.title)
```

Output — Targets: index.html line 12

```
Salary guide - DataJobs
Data Analyst $92,000
DataJobs - Jobs for data people
```

To open a new empty tab yourself, use `driver.switch_to.new_window("tab")`.

12 Screenshots

Code	Saves
`driver.save_screenshot("page.png")`	the visible part of the page (the viewport). Returns `True`.
`el.screenshot("job2.png")`	one element only
`driver.get_screenshot_as_png()`	nothing; returns the PNG bytes

```
print(driver.save_screenshot("page.png"))          # visible area
card = driver.find_element(By.ID, "job-2")
print(card.screenshot("job2.png"))                 # one element
```

Output — Targets: index.html lines 30--34

```
True
True
```



Figure: the real file `job2.png` from the example above.

Screenshots help to debug headless runs. Take one when a locator fails.

13 Scraping workflow: Selenium + pandas

1. Open the page.
2. Make the page show all data (click, scroll).
3. Wait for the data.
4. Read each item into a dictionary.
5. Make a `DataFrame`. Clean the types.
6. Save it (CSV) and quit the driver.

```
import pandas as pd
driver.find_element(By.ID, "load-more").click()
WebDriverWait(driver, 5).until(
    lambda d: len(d.find_elements(By.CSS_SELECTOR, "li.job")) == 7)

rows = []
for card in driver.find_elements(By.CSS_SELECTOR, "li.job"):
    rows.append({
        "title":   card.find_element(By.CSS_SELECTOR, ".job-title a").text,
        "company": card.find_element(By.CLASS_NAME, "company").text,
        "city":    card.find_element(By.CLASS_NAME, "location").text,
        "salary":  int(card.get_attribute("data-salary")),
    })
df = pd.DataFrame(rows)
print(df.sort_values("salary", ascending=False).to_string(index=False))
print(df.groupby("city")["salary"].mean().round().head(3))
df.to_csv("jobs.csv", index=False)
```

Output — Targets: index.html lines 25--44, plus 3 added by script lines 89--103

```
             title         company      city  salary
Research Scientist   Quanta Health    Remote  140000
    Data Scientist       Nimbus AI    Remote  130000
 Analytics Manager       Kite Bank    Mumbai  125000
       ML Engineer      Orbit Labs Bengaluru  120000
     Data Engineer Delta Logistics   Chennai  110000
      Data Analyst     Acme Retail      Pune   95000
        BI Analyst       Kite Bank    Mumbai   88000
city
Bengaluru    120000.0
Chennai      110000.0
Mumbai       106500.0
Name: salary, dtype: float64
```

Hand the page to BeautifulSoup

After the waits, `driver.page_source` has the full HTML. BeautifulSoup parses it fast. Selenium makes one call to the browser per `.text` or `find_element`; BeautifulSoup makes none.

```
from bs4 import BeautifulSoup
import pandas as pd
driver.find_element(By.ID, "load-more").click()
WebDriverWait(driver, 5).until(EC.presence_of_element_located((By.ID, "job-7")))

soup = BeautifulSoup(driver.page_source, "html.parser")   # hand over the HTML
df2 = pd.DataFrame([{
    "title":  li.select_one(".job-title a").get_text(strip=True),
    "salary": int(li["data-salary"]),
} for li in soup.select("li.job")])
print(df2.shape)
print(df2.tail(3).to_string(index=False))
```

Output — Targets: same li.job elements, read from page_source

```
(7, 2)
             title  salary
     Data Engineer  110000
Research Scientist  140000
 Analytics Manager  125000
```

Scrape politely

Read the site's terms and `robots.txt`. Add pauses between page loads on real sites. Prefer an official API or a download if the site has one.

14 Common pitfalls

Problem	Cause	Fix
`NoSuchElementException`	Wrong locator, element not loaded yet, or element inside an iframe	Check the locator in the browser DevTools. Use `WebDriverWait`. Switch to the frame.
`StaleElementReferenceException`	The page or the element was rebuilt after you found it	Find the element again after the change.
`ElementNotInteractableException`	The element is hidden or has no size	Wait for `element_to_be_clickable`. Scroll to it.
`ElementClickInterceptedException`	Another element covers it (banner, popup)	Close the popup. Scroll the element into view.
`TimeoutException`	The wait condition never became true	Check the condition and the locator. Raise the timeout only if the site is slow.
`InvalidSelectorException`	Bad CSS or XPath; spaces in `By.CLASS_NAME`	Fix the syntax. Use `By.CSS_SELECTOR` for several classes.

```python
title = driver.find_element(By.CSS_SELECTOR, "#job-1 a")
driver.refresh()                    # the page is rebuilt
try:
    print(title.text)
except StaleElementReferenceException as e:
    print(type(e).__name__)
title = driver.find_element(By.CSS_SELECTOR, "#job-1 a")   # find it again
print(title.text)
```

Output — Targets: index.html line 26

```
StaleElementReferenceException
Data Analyst
```

```python
driver.find_element(By.ID, "search").send_keys("scientist", Keys.ENTER)
a = driver.find_element(By.CSS_SELECTOR, "#job-1 a")      # now hidden
print(a.is_displayed(), repr(a.text))
print(repr(a.get_attribute("textContent")))
a.click()
```

Output (raises an error; message cut to its first line) — Targets: index.html line 26

```
False ''
'Data Analyst'
ElementNotInteractableException: element not interactable
```

```
print(len(driver.find_elements(By.CSS_SELECTOR, ".job.remote")))
print(len(driver.find_elements(By.CLASS_NAME, "job remote")))  # two classes: wrong
```

Output (raises an error; message cut to its first line) Targets: index.html lines 30--34

```
1
InvalidSelectorException: Compound class names are not allowed.; For documentation on this error,
↪  please visit:
↪  https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#invalidselectorexception
```

14.1 Headless differences

Headless Chrome uses a small default window. Pages can show a mobile layout or hide menus at small sizes. Set the window size:

```
options = webdriver.ChromeOptions()
options.add_argument("--headless=new")      # no --window-size this time
d = webdriver.Chrome(options=options)
print(d.get_window_size())
d.quit()
```

Output Targets: No page needed

```
{'width': 780, 'height': 580}
```

Check	Action
Layout differs from your normal browser	Set `--window-size=1200,800` or larger.
A step fails only in headless mode	Take a screenshot at that step. Run once without `--headless=new` to watch.
Browser processes stay open	Call `driver.quit()` in a `finally` block.
Code works once, fails the next run	Timing problem. Replace `time.sleep` with an explicit wait.

15 Cheat sheet

The rows run in order on one fresh page. The result column is the real return value. A → shows the value of a check expression after the step.

Task	Code	Real result
Pages		
Open a page	`driver.get(URL)`	`None`
Page title	`driver.title`	`'DataJobs - Jobs ...`
Back, then forward	`driver.back(); driver.forward()`	→ `'DataJobs - Jobs ...`
Close the browser	`driver.quit()`	`(not run)`
Locate		
One element	`driver.find_element(By.ID, "search")`	→ `'input'`
All matches (list)	`driver.find_elements(By.CLASS_NAME, "job")`	→ `4`
CSS selector	`driver.find_element(By.CSS_SELECTOR, "#job-2 .salary")`	→ `'$130,000'`
Link text	`driver.find_element(By.LINK_TEXT, "About")`	→ `'about.html'`
Search inside an element	`driver.find_element(By.ID, "job-3").find_element(By.CLASS_NAME, "company")`	→ `'Orbit Labs'`
Read		
Visible text	`driver.find_element(By.ID, "result-count").text`	`'Showing 4 jobs'`
Attribute or property	`driver.find_element(By.ID, "job-1").get_attribute("data-salary")`	`'95000'`
Attribute as in the HTML	`driver.find_element(By.CSS_SELECTOR, "#job-1 a").get_dom_attribute("href")`	`'jobs/1.html'`
Checkbox ticked?	`driver.find_element(By.ID, "remember").is_selected()`	`False`
Interact		
Type text	`driver.find_element(By.ID, "search").send_keys("ml")`	→ `'ml'`
Press a key	`driver.find_element(By.ID, "search").send_keys(Keys.ENTER)`	→ `'Showing 1 jobs'`
Clear a field	`driver.find_element(By.ID, "search").clear()`	→ `''`
Click	`driver.find_element(By.ID, "remember").click()`	→ `True`
Choose in a dropdown	`Select(driver.find_element(By.ID, "role")).select_by_visible_text("Recruiter")`	→ `'Recruiter'`
Submit a form	`driver.find_element(By.ID, "username").submit()`	→ `'Enter a username'`
Wait		
Explicit wait	`driver.find_element(By.ID, "load-more").click(); WebDriverWait(driver, 5).until(EC.presence_of_element_located((By.ID, "job-5")))`	→ `'job-5'`
Implicit wait (global)	`driver.implicitly_wait(5)`	→ `5.0`
JavaScript, alerts, frames, tabs		
Run JavaScript	`driver.execute_script("return document.title.length")`	`31`
Scroll to the bottom	`driver.execute_script("window.scrollTo(0, document.body.scrollHeight)")`	`None`
Accept an alert	`driver.find_element(By.ID, "alert-btn").click(); driver.switch_to.alert.accept()`	→ `'DataJobs'`
Enter, then leave an iframe	`driver.switch_to.frame("news"); driver.switch_to.default_content()`	→ `'DataJobs'`
Screenshot of the page	`driver.save_screenshot("page.png")`	`True`
HTML for BeautifulSoup	`driver.page_source`	→ `'<html lang="en"'`

Imports: `By` (`selenium.webdriver.common.by`), `Keys` (`...common.keys`), `Select`, `WebDriverWait` (`...support.ui`), `EC` (`...support.expected_conditions`).

A Glossary

Term	Meaning
WebDriver	The W3C standard API that controls a browser. Also the Python object `driver`.
Browser driver	A program that receives WebDriver commands for one browser: `chromedriver`, `geckodriver`, `msedgedriver`.
Selenium Manager	The tool in Selenium that finds or downloads the browser driver and browser.
Headless	A browser that runs with no visible window.
DOM	The tree of elements that the browser builds from HTML. JavaScript can change it.
`WebElement`	The Python object for one element in the page.
Locator	A pair of strategy and value, for example `(By.ID, "search")`.
CSS selector	A pattern that matches elements by tag, id, class and attributes.
XPath	A path language for nodes in an XML or HTML tree.
Implicit wait	A global timeout for every `find_element` call.
Explicit wait	A wait for one condition: `WebDriverWait(...).until(...)`.
Expected condition	A ready-made condition for an explicit wait (`EC.…`).
Stale element	A `WebElement` whose element is no longer in the page.
Iframe	A page shown inside another page.
Window handle	The ID string of one browser window or tab.
`page_source`	The current HTML of the page, as text.

Sources

- Selenium documentation: `https://www.selenium.dev/documentation/` (WebDriver, Waits, Selenium Manager, Browser options, Actions API).
- Selenium with Python (unofficial API guide): `https://selenium-python.readthedocs.io/`.
- Source and docstrings of the installed `selenium` 4.51.0 package (`get_attribute`, `WebDriverWait` defaults).